

RLHF (reinforcement learning from human feedback)

Normally, we start with

pre-training



SFT (Supervised fine-tuning)

(using high quality data, if we are building a chat bot, we also prepared the training samples as Q - A format)

note that after pretraining, the model only learns complete texts, this is different from answering a question.

e.g. a question could be followed by another question.



RLHF

Stage 1: training a reward function (using human feedback)

r_θ usually a fine-tuned LLM

feed a prompt into fine-tuned model

answer A:
answer B
answer C

} ranked by human
e.g. A > B > C

loss for pair A and B

$$\text{loss} = -\log(r_\theta(A) - r_\theta(B))$$

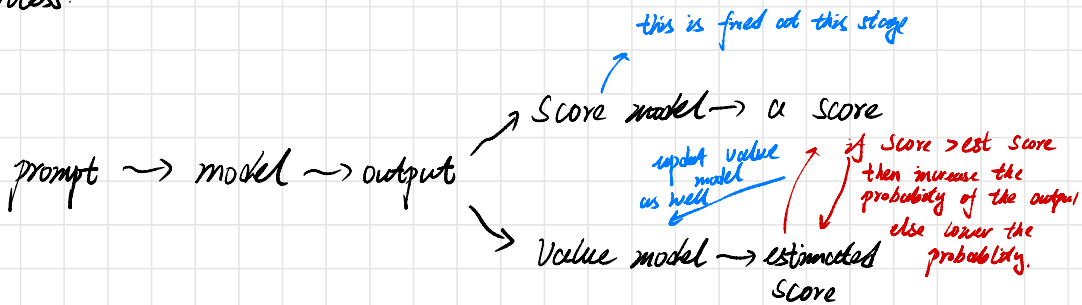
after training the reward model, we can evaluate the output of an answer given by LLM, e.g. 4.2, but we can't tune the model using this score!

This is why we need RL

Stage 2: proximal policy optimization (PPO)

We need a value model v_ϕ to estimate the score of an output of LLM
(normally a copy of our model after SFT)

process:



$$\Delta = \text{advantage} = \text{Score} - \text{estimated Score}.$$

Vanilla policy gradient:

Compute the gradient of weighting (by treating the answer to be correct)

$$\text{then } \text{weights} = \text{weights} - \Delta \cdot \text{gradients}$$

problem: change the gradient too much
if $\Delta > 0 \rightarrow$ increase the probability
catastrophic forgetting if $\Delta < 0 \rightarrow$ decrease the probability

PPO:

(clip the gradient of policy gradient descent)

probability ratio: for a word w

$$r = \frac{\text{prob}(w \text{ after updating according to policy gradient})}{\text{prob}(w \text{ before updating})}$$

the weights after being should make $1 - \epsilon < r < 1 + \epsilon$